VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
SOFTWARE ENGINEERING

GLAMMorous exchange

Stock market architecture

Software design and architecture project

Author: 3rd course 3rd group students
Gediminas Pakšys
Martynas Deksnys
Martynas Vasiliauskas
Aušris Navaslauskas
Laurynas Bražas

Supervisor: lekt. Vasilij Savin

Vilnius – 2026

Contents

1. INTRO 3
   1.1. Overview 3
   1.2. Core audience 3
   1.3. Business problem 3
2. BUSINESS REQUIREMENTS 4
3. BUSINESS FLOWS 5
   3.1. General business rules 5
   3.2. General business constraints 6
   3.3. Searching for a stock 7
   3.4. Login process 7
   3.5. Making a market order 8
      3.5.1. Main scenario: 8
      3.5.2. Alternative scenario: 9
   3.6. Making a limit order 10
   3.7. Creating a stock 12
   3.8. Viewing stock price history 13
   3.9. Adding funds and automatic currency conversion 14
   3.10. Simulating user activity (optional) 15
4. NON-FUNCTIONAL REQUIREMENTS 16
5. HIGH LEVEL ARCHITECTURE 17
   5.1. Client 17
      5.1.1. Optional 17
   5.2. Server 18
      5.2.1. Optional 19
   5.3. Protocol 19
6. UI/UX WIREFRAMES 20
   6.1. Register page 20
   6.2. Login page 20
   6.3. Sidebar 20
   6.4. Portfolio page 20
   6.5. Deposit form 21
   6.6. Withdrawal form 21
   6.7. Company dashboard page 21
   6.8. Apply to list a stock form 21
   6.9. Stock preview page 21
   6.10. Place an order form 22
   6.11. Settings page 22
   6.12. Admin dashboard (optional) 22
7. DATA MODEL 23

1. Intro

1.1. Overview

GLAMMorous exchange will be a desktop application that provides a platform for users to buy and sell stocks through a centralized exchange. Users will be able to deposit and withdraw cash, browse available stocks, view market prices and price charts, manage their portfolios, and monitor market activity through an interactive interface.

The exchange will support market and limit orders. Users will be able to submit buy and sell orders, which will be validated and placed into an order book. The system will then match compatible buy and sell orders and execute them according to system rules. Successful trades will update users' portfolios and corresponding market prices.

GLAMMorous exchange will provide the core functionality required to facilitate stock trading, including order management, trade execution, portfolio management, and market monitoring.

1.2. Core audience

The primary audience of *GLAMMorous exchange* will be:

- **Individual investors** who want to buy and sell stocks and manage their investments through a single platform.
- **Active traders** who need to track market prices, manage pending orders, and respond to changes in market conditions.
- **New market participants** who want an accessible platform to enter the stock market and manage their initial investments.
- **Companies** who want to issue their own stocks, and make them available to investors.

1.3. Business problem

Participating in the stock market requires users to access market information, submit buy and sell orders, monitor and manage their investments. These activities require a platform that connects trading functionality and portfolio management together in a clear and accessible interface.

Users need a reliable way to deposit and withdraw money, submit orders, see their holdings, and monitor changes in stock prices. Without a centralized system for all these activities, managing trades and keeping track of investments would be more complicated and less efficient.

GLAMMorous exchange will address this problem by providing a centralized platform for stock trading and managing investments. Users will be able to submit market and limit orders, see the order book, monitor pending and completed orders, track their portfolios. The exchange will process compatible orders according to defined system rules, providing a consistent and transparent approach to trade execution.

2. Business requirements

The MoSCoW framework is used to define priorities of certain system features.

Must:

- Run as a native desktop application.
- Support at least one major desktop platform (Windows, macOS, or Linux).
- Handle user inputs (keyboard and mouse).
- Provide secure authentication for trader and company accounts.
- Enable company accounts to issue and list new stocks.
- Search listed stocks.
- Display real-time stock prices and fundamental market data.
- Execute market and limit orders.
- Display portfolio.
- Render historical stock price charts.
- Handle deposit and withdrawal requests for account funds.
- Track personal order history (pending and closed orders).
- Allow manual cancellation of active limit orders.
- Automatically expire unfulfilled limit orders after a set duration (maximum 90 days).

Should:

- Switch between stock price timeframes (e.g., 1 hour, 1 day, 1 week).
- Track profit/loss and percentage changes for individual stocks and the overall portfolio.
- Support at least two currencies.
- Select a primary currency.
- Convert user funds into the currency required for a stock purchase.

Could:

- Place stop orders.
- Receive price alert notifications when a stock hits a target threshold.
- Export portfolio reports (e.g., PDF format).
- Support multiple operating systems (Windows, macOS, and Linux).
- Purchase fractional shares of stocks.
- Simulate background user activity.
- Customize client settings (e.g., font size).
- Provide admin accounts to review, approve, or reject company stock creation requests prior to trading.

Would not:

- Support real-money transactions.
- Offer personalized investment advice.
- Allow users to view, search, or follow other user accounts.
- Execute automated algorithmic trading or custom strategies.

3. Business flows

3.1. General business rules

Market order execution: a market order must be matched immediately against available opposite orders in the order book, prioritizing the best prices. If multiple orders have the same price, the oldest order must be matched first.

Partial market order execution: if there aren't enough opposite orders to fulfil a market order completely, the system must offer the user the option to execute the order with the available quantity. The unfulfilled remainder is cancelled.

Limit order execution: a limit order remains in the order book until it is fully fulfilled, cancelled by the user, or expires.

Limit order price: a buy limit order can only be matched at a price equal to or lower than its specified limit price. A sell limit order can only be matched at a price equal to or higher than its specified limit price.

Order priority: orders with more favorable prices must have priority over less favorable orders. Orders with identical prices must be processed in chronological order.

Reservation of funds and shares: when a limit order is accepted, the required cash or shares must be reserved. Reserved assets cannot be used for other transactions until the order is fulfilled, cancelled, or expires.

Release of reserved assets: when a limit order is cancelled or expires, any remaining reserved cash or shares must be returned to the user's balance or holdings.

Creating a stock: a company account can create stocks associated with that company. Creating a stock assigns the issued shares to the company's portfolio and places a limit sell order at the specified initial price.

Stock approval (optional): if admin accounts are enabled, newly created stocks cannot be traded until an administrator approves it.

Order history: every order must have a closed order record, regardless of whether it was fulfilled, cancelled, or expired. Partially fulfilled orders must retain their executed quantity and transaction value.

Current stock price: the current stock price represents the price of the most recently executed trade.

Historical stock prices: historical price chart values must be calculated using the volume-weighted average price of trades within each time interval.

Account permissions: only company accounts may create stocks. Users may only access and modify their own account balances, portfolios, and orders. Administrator actions are restricted to administrator accounts.

Portfolio valuation: the total portfolio value will be displayed in the user's primary currency. It will sum up available cash balances and invested assets, converting the secondary currency values to the primary currency using the exchange rate.

Invested value: the total invested value will be displayed in user's primary currency. Stocks traded in a different currency wil be converted using the exchange rate.

Currency conversion when buying stocks: the system must ensure that sufficient funds are available in the stock's trading currency. If the user has insufficient funds in that currency, but sufficient funds in another, the system must convert the required amount before executing the trade.

Currency of sale proceeds: proceeds from selling a stock must be credited to the user's account in the currency in which the stock is traded.

Primary currency selection: users may select either EUR or USD as their primary currency in application settings. Changing the primary currency affects portfolio valuation and the currency used for deposits and withdrawals, but does not automatically convert existing account balances.

Currency consistency: all transactions must be executed using the currency in which the stock is traded. Account balances must be updated in corresponding currencies.

Deposits and withdrawals: deposits and withdrawals will be performed in the user's currently selected primary currency. If the user has insufficient funds in their primary currency when requesting a withdrawal, the system must convert the required amount from their secondary currency, if sufficient funds exist.

3.2. General business constraints

- A limit order cannot have an expiration period longer than 90 days;
- Stock ticker symbols must consist of uppercase English letters;
- Stock ticker symbols must be unique;
- The number of shares issued when creating a stock must be positive;
- The initial stock price must be positive;
- Order quantity must be positive;
- User cannot sell more shares than they have available;
- User cannot spend more money than their balance allows;
- User cannot create stocks without a company account;
- Stock must be approved by administrator before it can be traded (optional);
- A market order cannot remain pending in the order book;
- User cannot use cash or shares reserved for pending limit orders;
- The main currencies in the application will be EUR and USD;
- Currency conversion must use exchange rates from the currency exchange service;
- Each user must have exactly one primary currency selected;
- Currency conversion cannot use funds reserved for pending orders;
- A transaction requiring currency conversion cannot be completed if a valid exchange rate is unavailable;
- Currency conversion must not result in a negative balance value.

3.3. Searching for a stock

1. The user enters a company name or stock symbol in the search field and submits the query.
2. The search request is sent to the server through ServerConnection.
3. StockCatalogService searches for stock records matching the entered query in the dataStore.
4. The matching stock records are retrieved and returned to the client.
5. If matching stocks are found, the stock name, the historic price chart, market price (the latest price a stock was bought at) are displayed.
6. If no matching stocks are found, the application informs the user that no results were found.

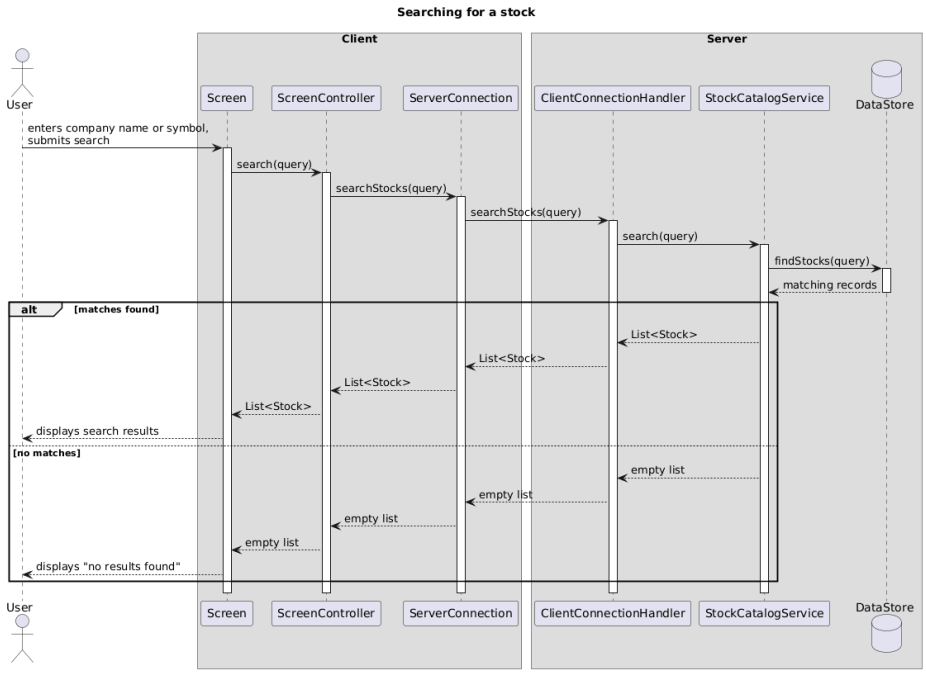


Figure 1. Sequence diagram for searching for a stock

3.4. Login process

1. The user enters either login or register information and presses the login button which sends a login request to the application.
2. The application validates that the input is not empty.
3. The application sends an authentication request to the server.
4. The authenticator queries the datastore for the account associated with the provided account.
5. The dataStore returns the user's account information and hashed password.
6. The authenticator verifies that the inputed and hashed passwords are the same.
7. If the credentials are invalid server rejects login and inform user that password is incorrect.

8. If the credentials are valid the server generates an authentication token JWT and sends a success response containing token and other relevant information.
9. The application stores the token and redirects user to the main application interface.

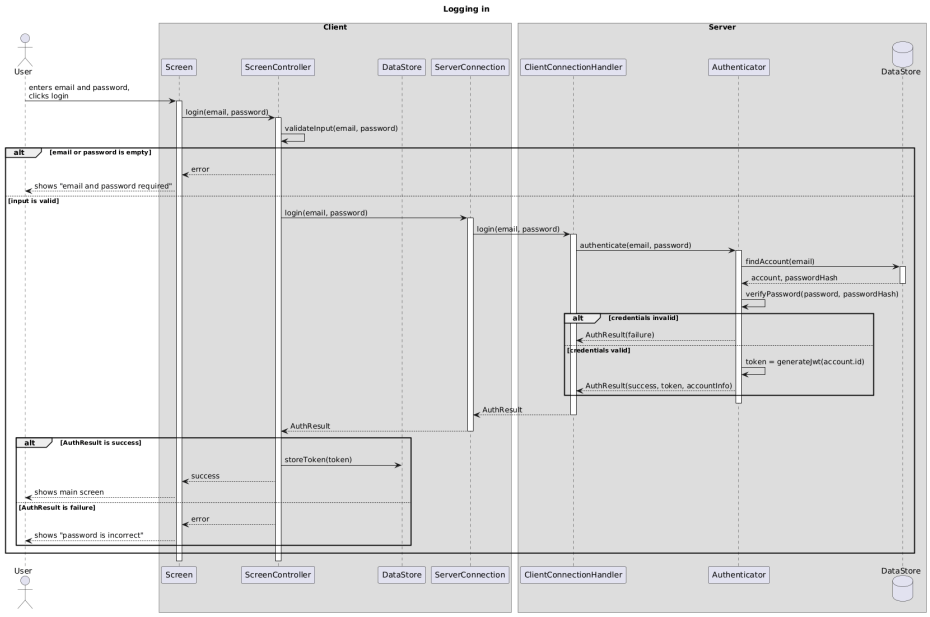


Figure 2. Sequence diagram for logging in

3.5. Making a market order

3.5.1. Main scenario:

1. The user selects Buy or Sell submits a market order.
2. The application sends the order to the server.
3. The server requests user information from the dataStore.
4. The Service validates the order's parameters and the user's permissions.
5. If validation succeeds the application requests market data from the dataStore and sends it to the Maching Engine.
6. The Matching Engine searches the order book for opposite orders and picks the most favourable ones; if there are multiple of same price the oldest one is picked.
7. The Account Service checks the user's available cash for a buy order or available stock holdings for a sell order.
8. If there are enough opposite orders, to fully fulfil the users order, the user is informed that his order was filled.
9. Currency exchange is performed if needed and the trade is executed.
10. The orders that the placed order was matched against are removed from OrderBook
11. The completed order is saved in Trade History.
12. The application sends the updated Account information – cash balances and stock holdings to the server which sends it to the datastore.

3.5.2. Alternative scenario:

6. If there aren't enough opposite orders, to fully fulfil the users order, the user gets a notification and the order amount is automatically changed to the floor from the available offers.
7. User is shown the total number of stock he will get and the total price if its a buy order. User is shown the total number of stocks he will sell and the total price if its a sell order.
8. User either cancels or confirms that he wants to proceed with the transaction.
9. The Account Service checks the user's available cash again for a buy order or available stock holdings for a sell order.
10. Currency exchange is performed if needed and the trade is executed.
11. The orders that the placed order was matched against are removed from OderBook
12. The completed order is saved in Trade History.
13. The application sends the updated Account information – cash balances and stock holdings to the server which sends it to the datastore.

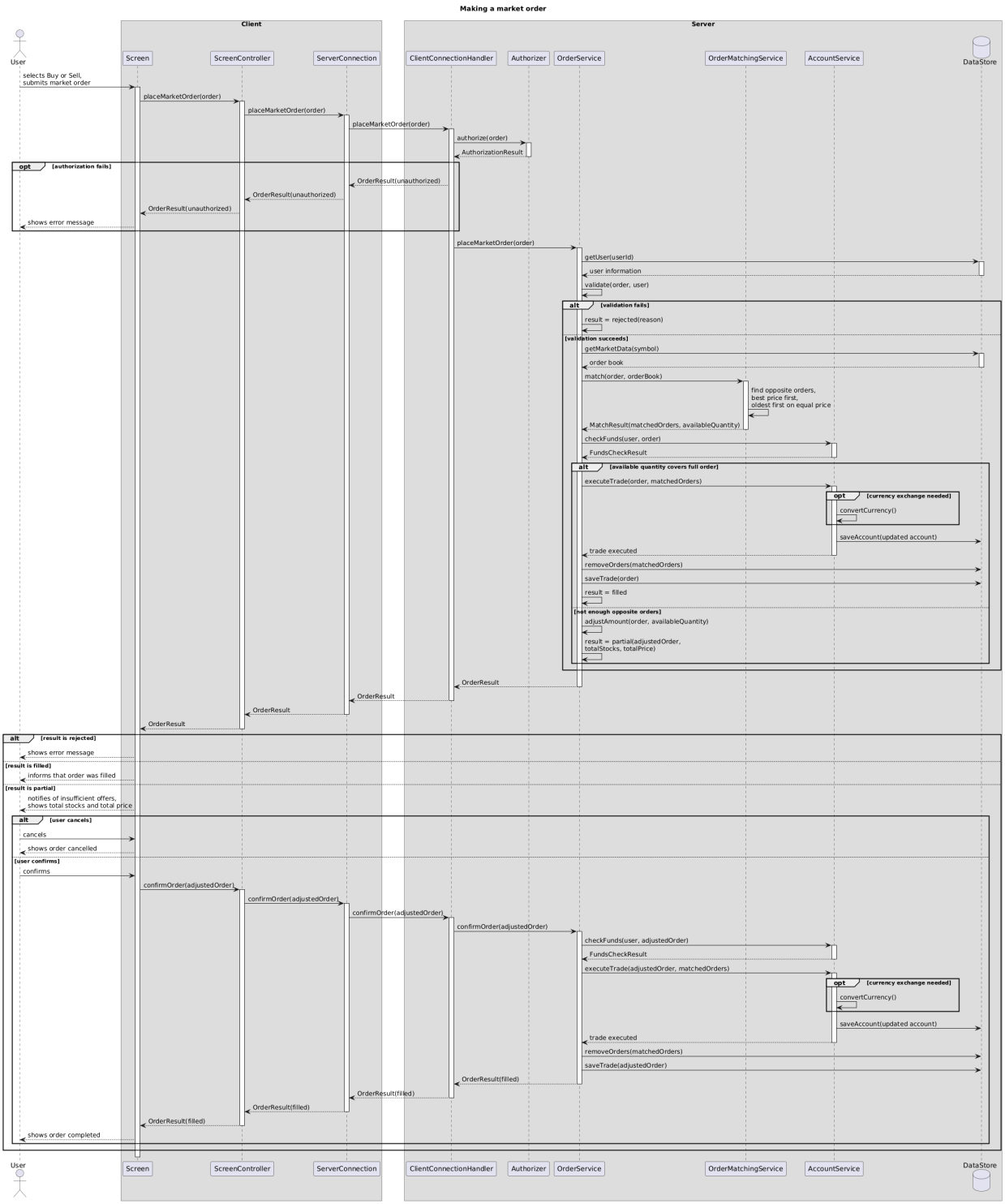


Figure 3. Sequence diagram making a market order

3.6. Making a limit order

1. The user selects Buy or Sell submits the price, quantity and expiration time of the limit order.
2. The application sends the order to the server.
3. The Service validates the order's parameters and the user's permissions.

4. The Account Service checks the user's available cash for a buy order or available stock holdings for a sell order.
5. If validation succeeds, the order is accepted, users money or stocks that he used for the trade are reserved (he owns them but can't use until the order is cancelled). The order is placed in the OrderBook and forwarded to the Matching Engine.
6. The Matching algorithm searches the order book for opposite orders that match the specified price until the order is fully completed or expires (at most 90 days).
7. When at least some part of the order is completed the user receives the stock/money. The information is placed into Trade History and OrderBook is updated.
8. When expiration time is reached or the order is canceled the order is removed from the OrderBook, any remaining reserved cash or stocks are released.
9. The application sends the updated Account information – cash balances and stock holdings to the server which sends it to the datastore.

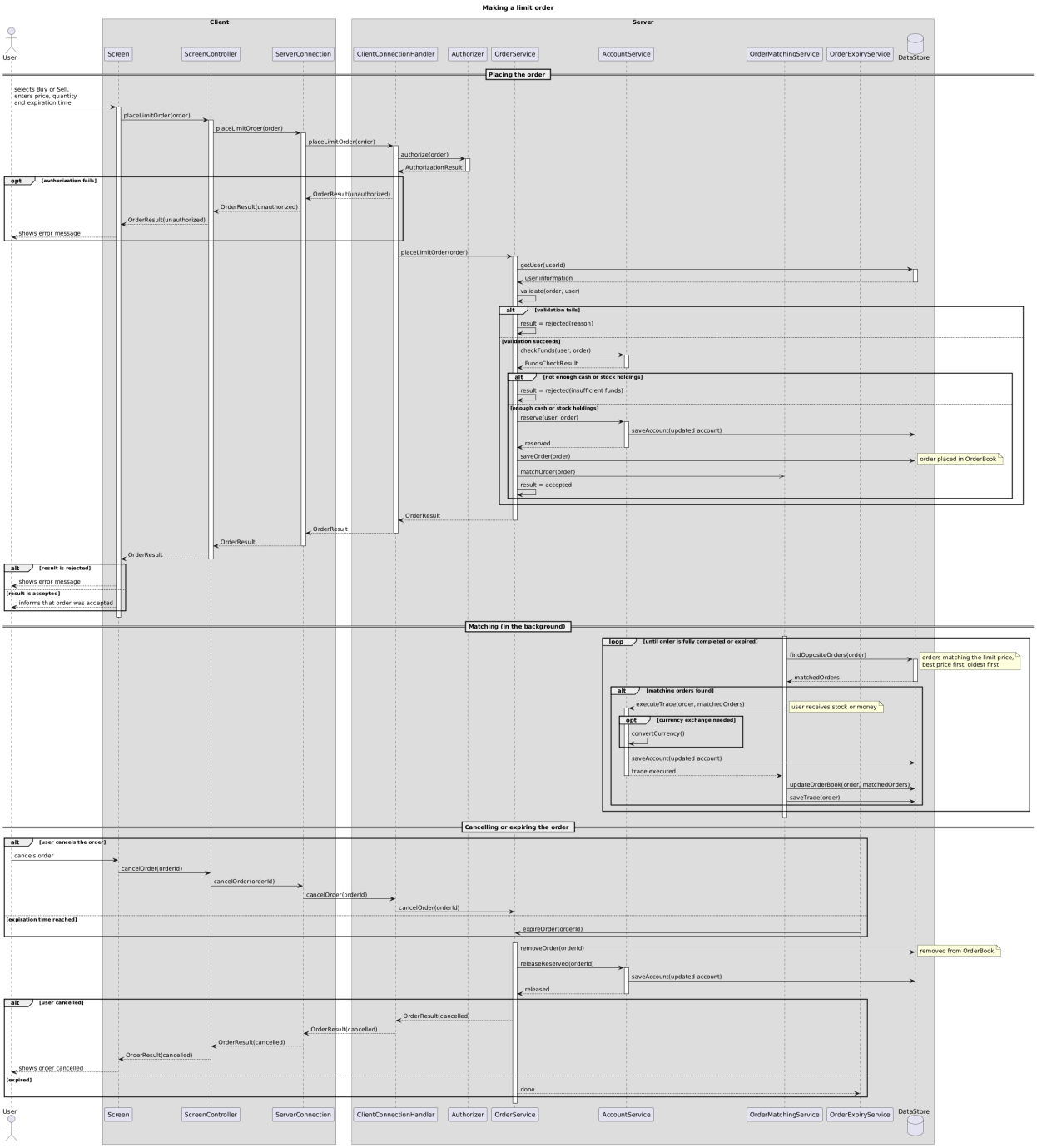


Figure 4. Sequence diagram for making a limit order

3.7. Creating a stock

1. The company sends a request to create a stock with name, ticker symbol, number of shares, initial stock price and which currency the stock will be traded at.
2. The application validates that the ticker symbol is comprised of 1-5 uppercase English letters (A-Z), the number of shares, initial stock price are positive numbers.
3. The application sends a request to the server to validate that the user is logged into a company account (has access), there isn't already a stock with the entered ticker symbol. The server gets the information to confirm against from the dataStore.

4. The application requests the information from the server.
5. If validation succeeds a stock is created and a sell limit order for the number of shares at a initial price is placed into the OrderBook, a success message is returned to the company.
6. All the information is sent to the DataStore.
7. If validation fails an error is returned.

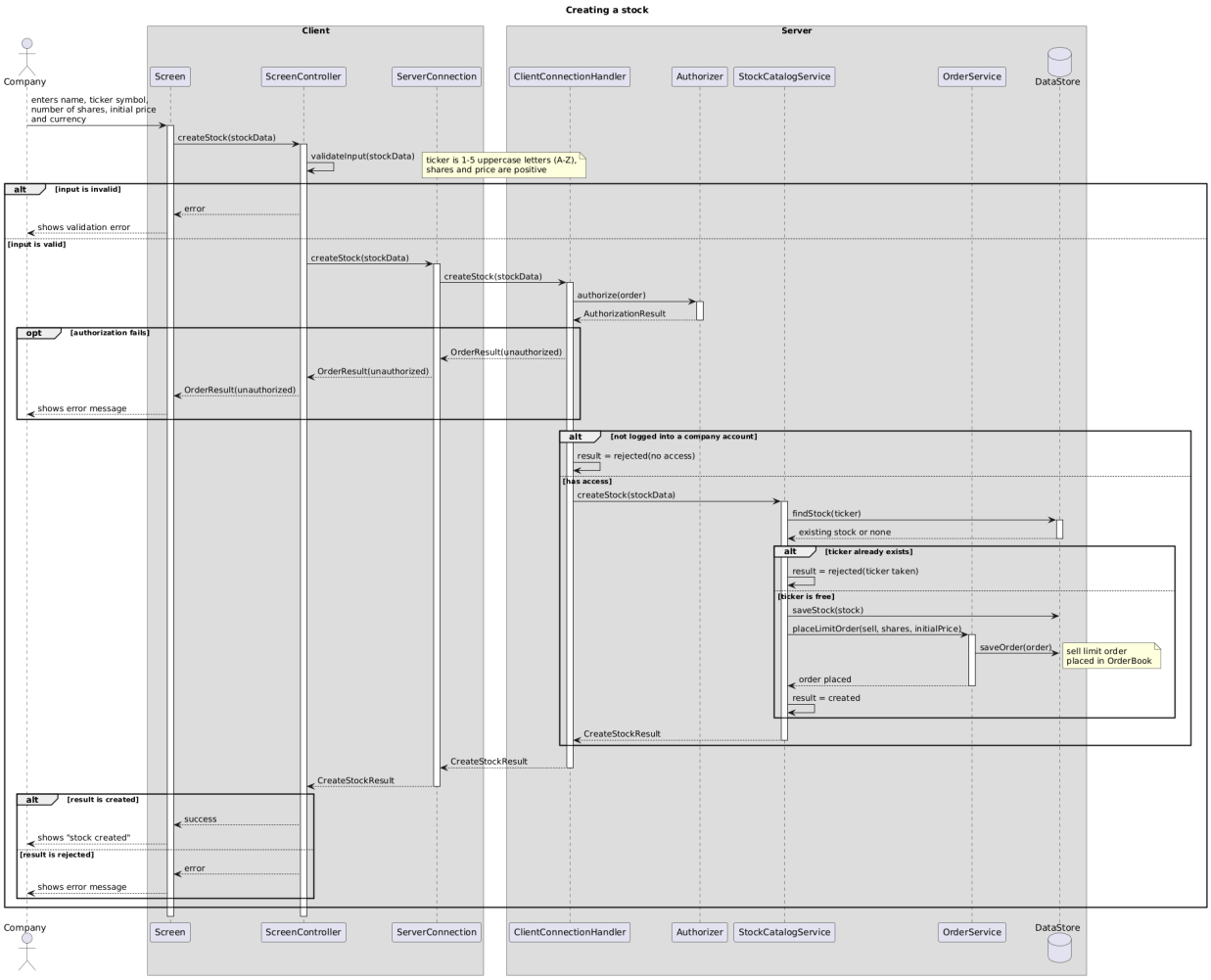


Figure 5. Sequence diagram for creating a stock

3.8. Viewing stock price history

1. The user selects a stock they want to view.
2. The application sends the request to the server.
3. The server gets the stock information from the dataStore.
4. The service retrieves completed trades for the requested historical period.
5. The service groups the completed trades into time intervals and calculates the VWAP (volume-weighted average price) for each interval.
6. If there were no trades in one of the middle intervals the point is skipped, if the latest interval is empty the previous point that had trades is displayed in its place.
7. If the user changes the chart period, the application requests historical data for the newly selected period.

8. The application updates the price chart.

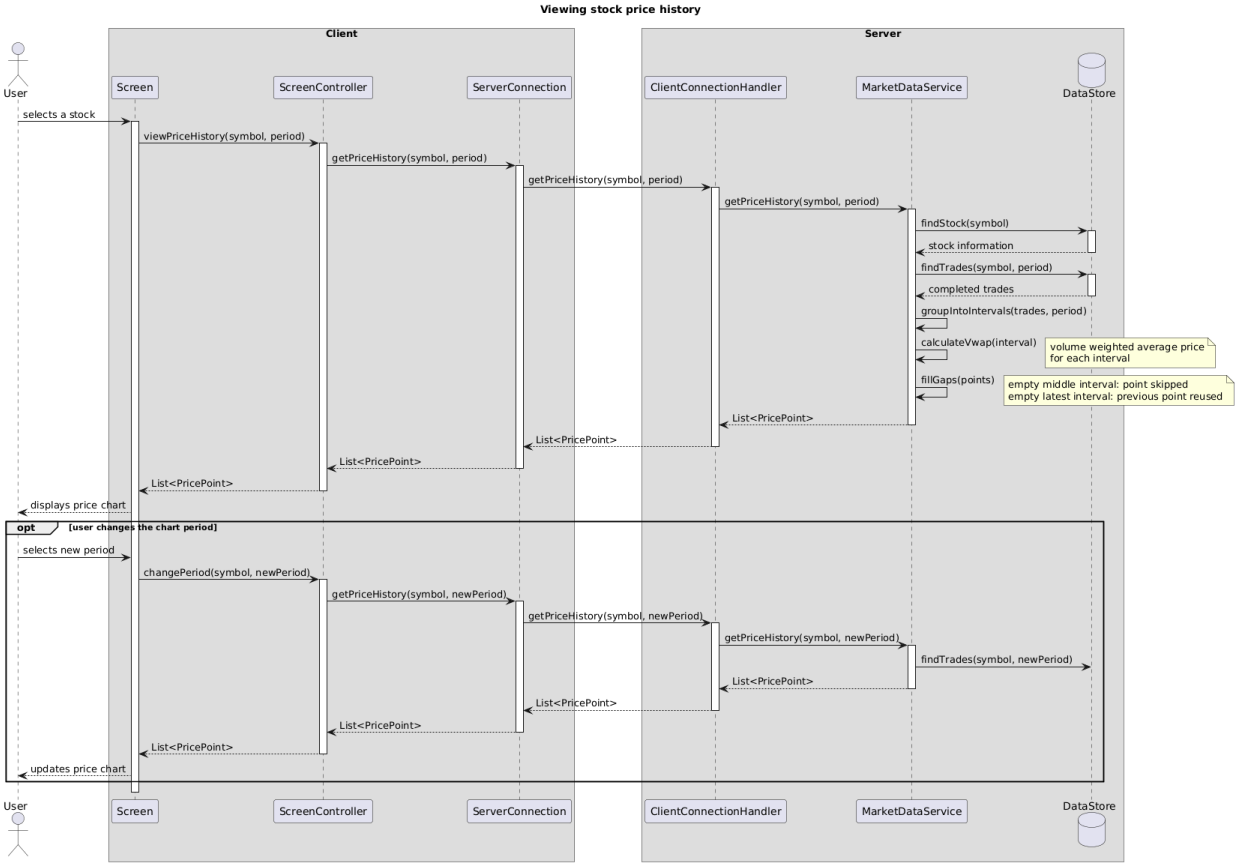


Figure 6. Sequence diagram for viewing stock price history

3.9. Adding funds and automatic currency conversion

1. The user chooses to add funds to his account (the funds can be various currencies).
2. The application sends the request to a financial service modeling platform (like Stripe developer, with test cards).
3. Financial service modeling platform checks the users cash and either approves the request or rejects it.
4. If the financial service modeling platform approves the request, the cash is sent to the user stock exchange account.
5. The user can now buy stocks, if the currency of the seller is the same that the buyer has the trade is executed normally.
6. If the currency of the seller is not the same that the buyer has, a request for automatic currency exchange is sent to the CurrencyService.
7. the CurrencyService sends a request to a currency exchange rate API (like Frankfurter) and performs the currency exchange and the trade is executed.

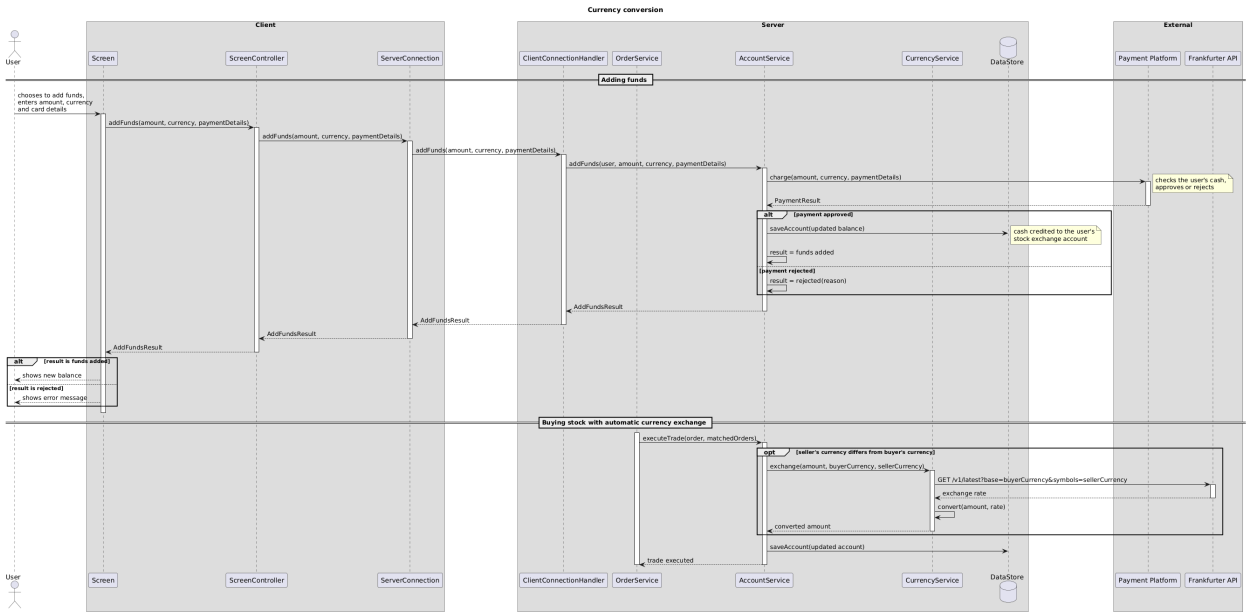


Figure 7. Sequence diagram for adding funds and automatic currency conversion

3.10. Simulating user activity (optional)

1. When user creates a stock MarketSimulator receives a request to simulate the activity for the stock.
2. MarketSimulator after a random interval creates a random limit order of random amount, random type (buy/sell), for random value (weighted around the current market price).
3. MarketSimulator sends the order to the stocks order book, and saves it in the DataStore.

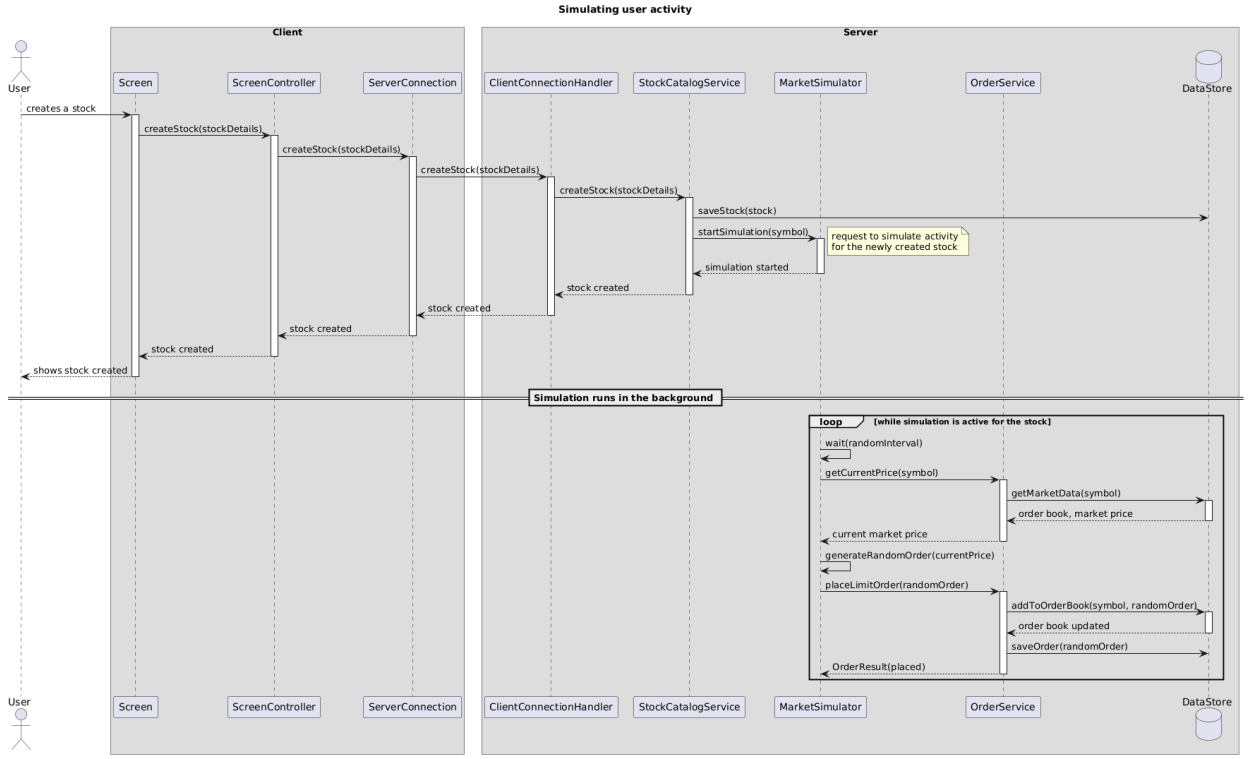


Figure 8. Sequence diagram for currency simulated user activity

4. Non-Functional Requirements

- **Performance:**
  - **Nfn-1.** 95% of order creation request should be completed within 1 second.
  - **Nfn-2.** 90% of deposit or withdraw requests should be completed within 5 seconds.
  - **Nfn-3.** 95% of requests to view current stock price and basic market data should be completed within 1 seconds.
  - **Nfn-4.** The application shall update displayed stock prices within 1 seconds of receiving new market data.
- **Reliability:**
  - **Nfn-5.** The system shall ensure that each accepted order is processed without unintended duplication.
  - **Nfn-6.** The system shall update account balances, executed orders, and portfolio holdings consistently when a trade is completed.
  - **Nfn-7.** The application shall display a clear error message when an operation fails and shall not report an unsuccessful operation as completed.
- **Usability:**
  - **Nfn-8.** A first-time user shall be able to locate a stock and navigate to its details within 2 minutes without external assistance.
  - **Nfn-9.** The application shall validate user inputs and provide an understandable explanation when an input is invalid.
  - **Nfn-10.** The application shall use consistent layouts, terminology, and interaction patterns across its main screens.
- **Security:**
  - **Nfn-11.** The application shall require users to authenticate before accessing account-specific information or performing account operations.
  - **Nfn-12.** The system shall authorize what users can access and modify based on their roles.
  - **Nfn-13.** User passwords shall never be stored in plaintext and shall be protected using a secure password-hashing.
  - **Nfn-14.** The system shall ensure that users can access and modify only their own portfolios, orders, and account balances.
- **Maintainability:**
  - **Nfn-15.** The system shall separate user-interface logic, business logic, and data-access logic into distinct components.

5. High level architecture

High level architecture of the whole system can be seen in figure 9.

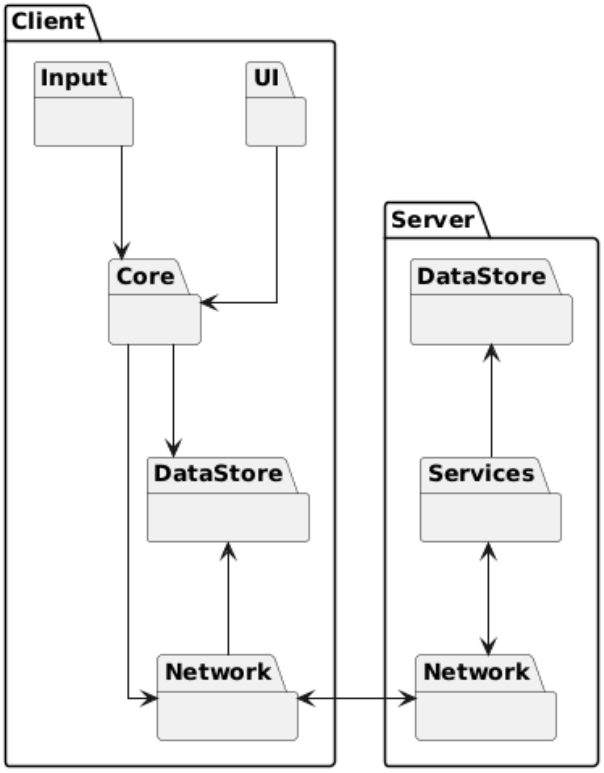


Figure 9. High level architecture of the whole system

5.1. Client

- **MouseInput** — delivers mouse events to ScreenController.
- **KeyboardInput** — delivers keyboard events to ScreenController.
- **Navigator** — holds the active page and its parameters, handles switching between pages.
- **Screen** — what the user sees (rendering). Shows the page that Navigator marks as active.
- **ScreenController** — handles input and user actions on that page, holds its state, validates input, talks to the server, and asks Navigator to switch pages.
- **ServerConnection** — handles the connection to the server and updates the DataStore with received data.
- **DataStore** — holds persistent local data (cached server data and the login token).

5.1.1. Optional

- Settings — local preferences (light/dark theme, UI scale, etc.), stored in the DataStore

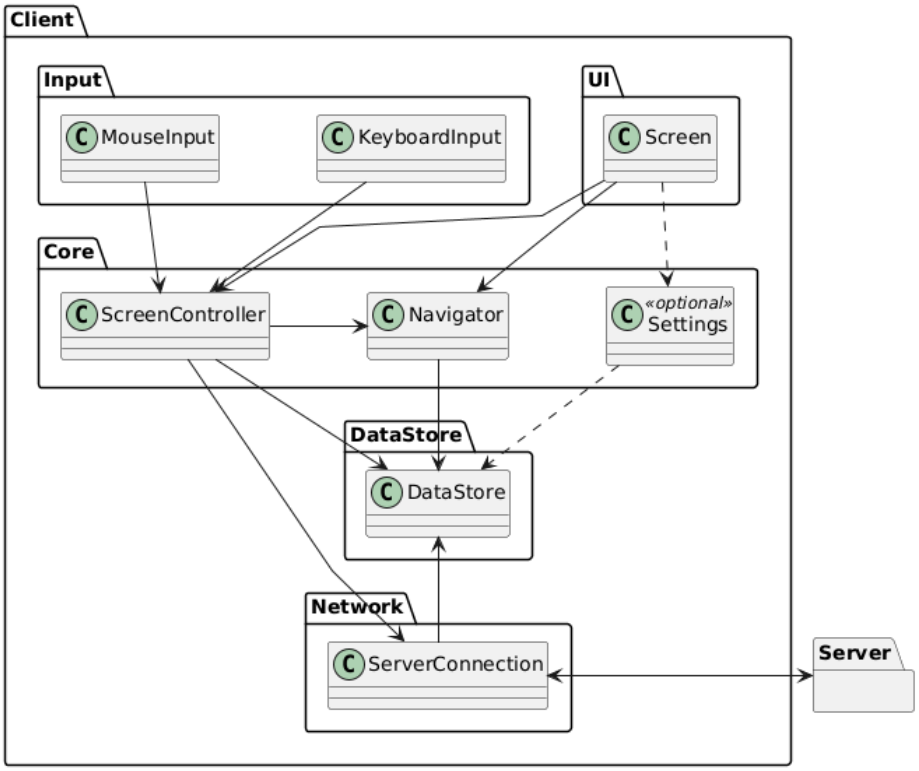


Figure 10. High level architecture for the client

5.2. Server

- **ClientConnectionHandler** — handles client connections, passes requests to services, and pushes updates to clients
- **Authenticator** — handles registration and login requests
- **Authorizer** — validates the login token and checks that the user's role (trader, company or admin) allows the requested action. For example, only company accounts may create stocks. The role to action rules are kept in one place
- **AccountService** — view account, add and withdraw funds, check available cash, reserve and release cash for pending orders. Only gives access to the user's own account
- **PortfolioService** — view user portfolio, profit / loss, check available holdings, reserve and release shares for pending orders. Only gives access to the user's own portfolio
- **OrderService** — validates placed orders, lists and cancels orders, reserves funds or shares, and passes accepted orders to matching. Only allows users to view and cancel their own orders
- **OrderMatchingService** — matches orders against open limit orders, executes trades, and creates closed orders
- **OrderExpiryService** — periodically finds open orders past their expiration time and closes them through OrderService
- **StockCatalogService** — stock company info, lookup, and stock creation by company accounts (gives the company its shares and places the initial sell order through OrderService, after approval if admin approval is enabled). Also lists the stocks of a company

- **MarketDataService** — provides current and historical prices and order book levels, and pushes price updates through ClientConnectionHandler
- **CurrencyService** — gets exchange rates from an API and converts between currencies
- **DataStore** — holds most data

5.2.1. Optional

- **MarketSimulator** — simulated users
- **NotificationService** — sends price alerts to clients
- **AdminService** — lets admin accounts approve or reject pending stock creation requests. Admin accounts are created manually and cannot be registered from the client
- **ReportService** — generates portfolio reports

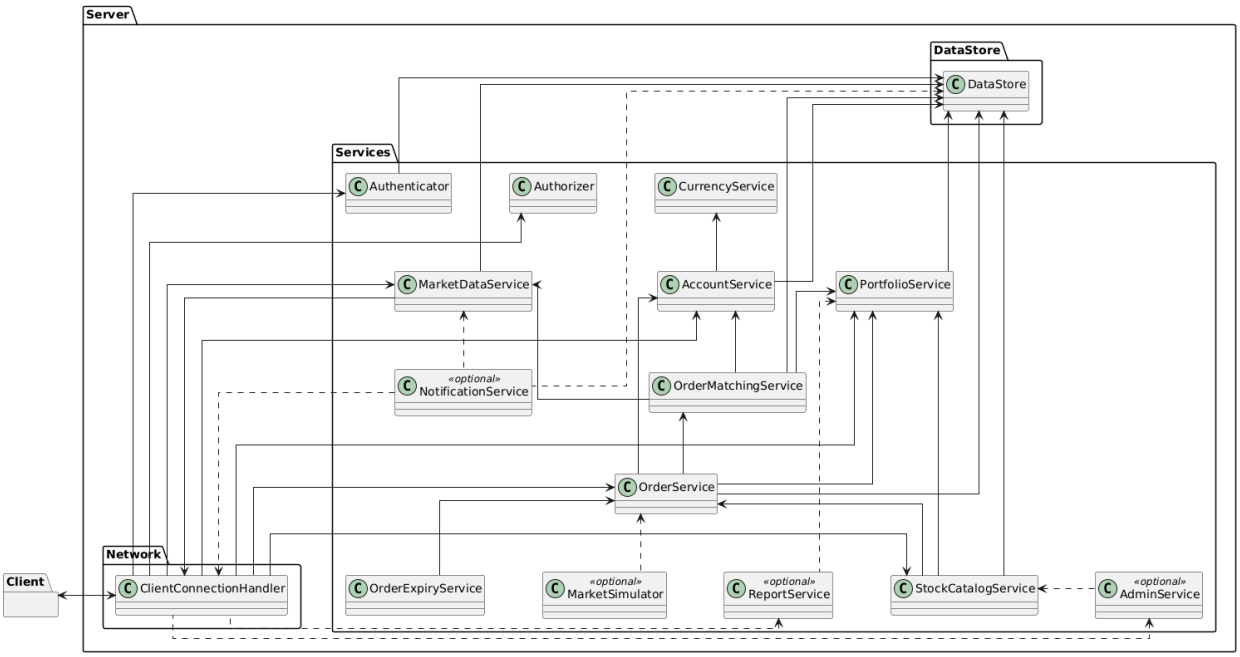


Figure 11. High level architecture for the server

5.3. Protocol

The client and the server communicate through messages sent between ServerConnection and ClientConnectionHandler. The client sends requests and receives responses. Every request after login carries the login token. The message format is not specified.

6. UI/UX wireframes

To reach the normal user and company interactive wireframe on figma, **click here**.

To reach the admin interactive wireframe on figma, **click here** (optional).

6.1. Register page

There will be separate registration forms for different account types.

Normal accounts will be required to enter their full name, email address, and a password.

Company accounts will be required to enter the company name, email address, and a password.

At the bottom of the register page, there will be a **Register** button, which will navigate to the portfolio or company dashboard based on the account type.

There will also be navigation options to the opposite registration form, and according login form.

6.2. Login page

There will be separate login forms for different account types.

Both **Normal accounts** and **Company accounts** will be required to enter the email address and password used during registration to login.

At the bottom of the login page, there will be a **Login** button, which will navigate to the portfolio or company dashboard based on the account type.

There will also be navigation options to the opposite login form, and according registration form.

6.3. Sidebar

The application will use a sidebar layout, which will be visible on every page, except the register and login pages. The sidebar is located on the left, and all page relevant information will be shown on the right. The sidebar will contain all available stocks. Clicking on a specific stock will bring up the **Stock preview** page. Below the sidebar, there are **Settings**, **Profile**, and **Logout** buttons, which bring up according pages.

6.4. Portfolio page

The portfolio page will only be reachable to normal accounts via the **Profile** button. It will be shown on the right of the sidebar. Values will be shown in the primary currency. At the top, total portfolio value is displayed, and the gain is highlighted. The total value is divided into available cash (not invested), and invested money. Below the available cash and invested value, cash balances are shown, containing the amount in EUR and USD. In the middle, holdings are shown. They contain the stock name, quantity of owned shares, total owned value, and gain/loss percentage. Holdings are sorted by value, from highest to lowest. Below the holdings, order

history is shown. Order history entries contain the stock name, order type, quantity, price, status, and also a button to cancel the order. Orders in the order history are sorted chronologically, from most recent to oldest.

There are **Deposit** and **Withdraw** buttons, which will open the money deposit, and money withdrawal forms accordingly.

6.5. Deposit form

This form will contain a field to enter the amount of money that is to be deposited. At the bottom of the form, there will be a **Confirm deposit** button, and in the top right of the form, there is an **X** to cancel.

6.6. Withdrawal form

This form will contain a field to enter the amount of money that is to be withdrawed. At the bottom of the form, there will be a **Confirm withdrawal** button, and in the top right of the form, there is an **X** to cancel.

6.7. Company dashboard page

The company dashboard will only be reachable to company accounts via the **Profile** button. It will be shown on the right of the sidebar. Values will be shown in the primary currency. At the top, total company market value will be shown, as well as shares outstanding (number of issued shares). Listed stocks are then shown, which contain the stock name, stock ticker symbol, current price per share, outstanding shares, and a 24 hour change percentage. Stocks are sorted by value, from highest to lowest. Below the stocks, there will be a pending stock list, where the stock name, ticker symbol, and status will be visible. At the very bottom, there is a **Apply to list a stock** button, that allows companies to apply to list new stocks.

6.8. Apply to list a stock form

This form will only be reachable to company accounts via the **Company dashboard** page. It will be a form, which accepts the stock name, ticker symbol, number of shares to issue and initial offering price. At the bottom of the form, there is a **Submit** button, and there is an **X** in the top right corner to cancel.

6.9. Stock preview page

The stock preview page is reachable by both normal and company accounts by clicking on a specific stock in the **Sidebar**. The stock preview shows a graph with historical data (each point is last traded price at different point in time). Below the graph, there are buttons **1H**, **1D**, **1W**, and **1M** used to select the timeframe. At the top right, the stock name and ticker is displayed. There is also a button **Place order**, which opens a place order form. Below the button, the order book is

shown. The order book shows 3 best buy orders (sorted from highest price to lowest) and 3 best sell orders (sorted from lowest to highest price). Below the order book, pending orders for the current stock can be seen. These use abbreviations **LB** - limit buy and **LS** - limit sell.

6.10. Place an order form

This form is opened by pressing the **Place order** button in the stock preview page. This form contains the stock name, allows selection of order type (market or limit), selection of side (buy or sell), as well as quantity. By default, the form is a market buy order.

Selecting **Limit** as the order type opens up two more fields: limit price (per share) and expiration date.

6.11. Settings page

The settings page will be reachable by both normal and company accounts by clicking on the settings icon below the sidebar. It will contain settings such as light/dark theme, UI scale, colorblind mode, primary currency, etc.

The default primary currency will be EUR.

6.12. Admin dashboard (optional)

The admin panel will have the amount of listed stocks displayed, as well as the amount of pending stock applications. Below the numbers, there will be a list of pending applications, with buttons **Accept** and **Reject** to confirm or deny the stock.

7. Data model

- **User** — an account with a login, a role (trader, company or admin), a cash balance per currency and a portfolio. Can create orders in the order book
- **UserRole** — says what kind of account a user has. Traders trade stocks, companies also create stocks, and admins (optional) approve stocks
- **Portfolio** — holds the stocks a user currently owns and their average buy price (used to calculate profit/loss). A company receives its issued shares here when it creates a stock
- **Stock** — a tradable security with its ticker, name, trading currency, issuing company and number of shares outstanding. Has an order book and a history
- **StockStatus** — (optional) says whether a stock is waiting for admin approval, approved or rejected. A stock can only be traded once approved
- **OrderBook** — holds the pending limit orders of one stock
- **StockHistory** — holds the records of one stock: executed trades and closed orders
- **Trade** — a record of one executed match between a buy and a sell order, with its quantity, price and time. Used for the current price, price history and charts
- **OrderSide** — says whether an order buys or sells
- **LimitOrder** — a pending order waiting in the order book for a matching order. It tracks how much is already fulfilled and the money exchanged so far
- **ClosedOrder** — the final record of a market or limit order that is no longer pending. It keeps what was requested (target quantity and price, with no price for market orders) and what was achieved (fulfilled quantity and the total money exchanged, which are both zero if nothing was fulfilled)
- **OrderOutcome** — says how a closed order ended: fulfilled, cancelled or expired

Market orders are never stored as open, since they are matched immediately. A limit order is pending while it is in openOrders. Every order (market or limit) ends as one ClosedOrder record with the same id, stored in the stock's history. An order that was cancelled or expired after some of it was fulfilled keeps its fulfilled quantity and value, so it counts as partly fulfilled. The average price is derived (fulfilledValue divided by fulfilledQuantity).

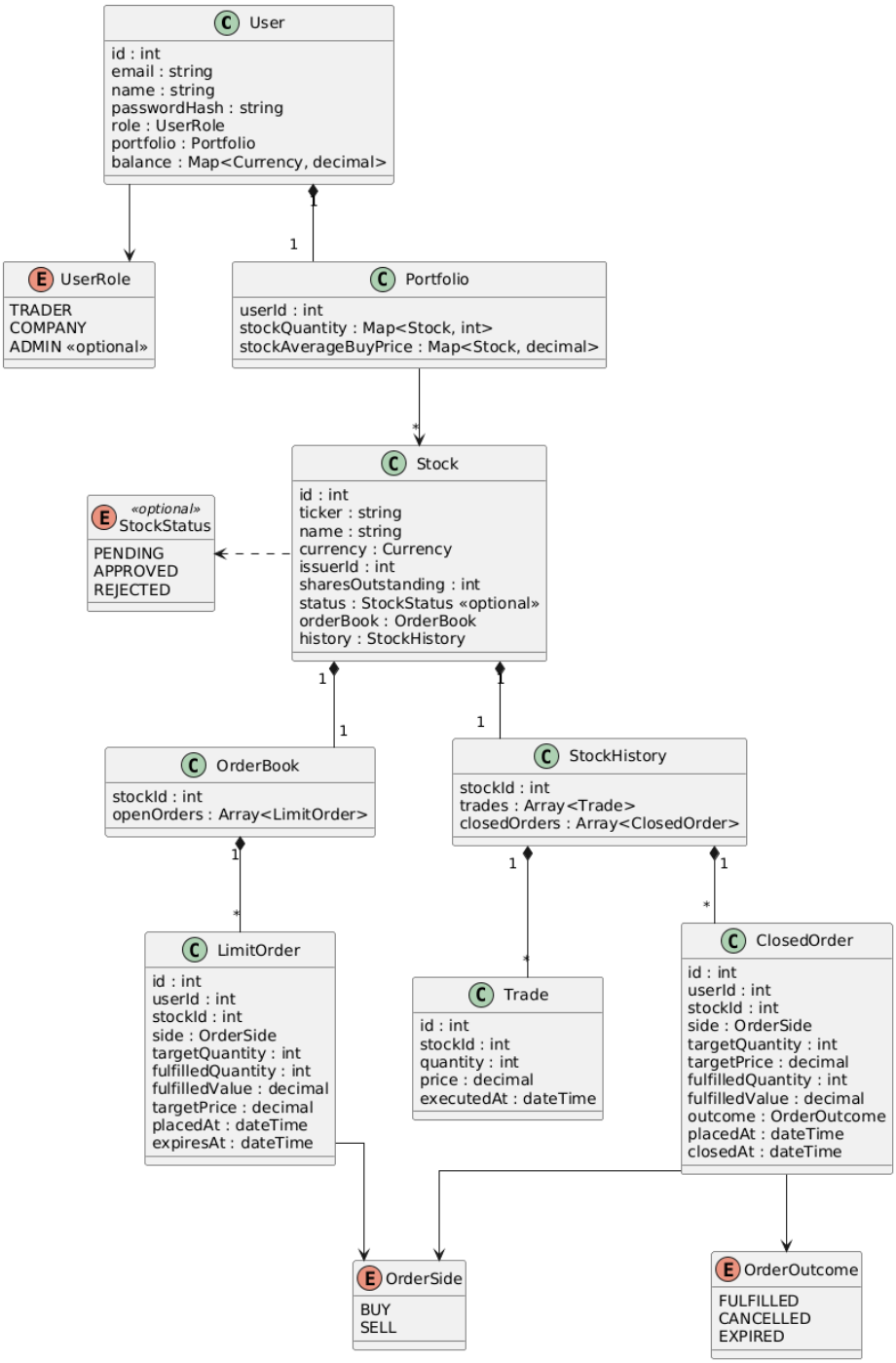


Figure 12. Data Model

Optional changes:

- **Fractional shares** — stock and order quantity change from `int` to `decimal`.
- **Admin** — add `ADMIN` role and `StockStatus`.